Создай файл AGENTS.md в корне проекта и зафиксируй правила работы для этого репозитория.

Содержимое AGENTS.md должно включать:

1. Цель проекта:
Прототип веб-приложения для управления задачами и дедлайнами (MVP), разработанный с помощью средств вайб-кодинга для лабораторной работы.

2. Стек:
- Next.js
- TypeScript
- Tailwind CSS
- Prisma
- SQLite

3. Принципы разработки:
- Не усложнять архитектуру (MVP, не production)
- Минимально достаточная декомпозиция
- Не использовать тяжёлые библиотеки без необходимости
- Избегать дублирования кода
- Не оставлять TODO и незавершённые куски

4. Правила работы агента:
- После значимых изменений всегда проверять:
  - установку зависимостей
  - сборку проекта
  - запуск dev сервера
- Исправлять ошибки сразу, а не откладывать
- Не останавливаться на полуреализации
- Обновлять README при изменениях

5. Требования к качеству:
- Код должен быть читаемым
- Логика должна быть понятной
- Проект должен быть пригоден для анализа в лабораторной работе

После создания AGENTS.md переходи к следующему этапу.

5 previous messages

Создан файл AGENTS.md в корне проекта и зафиксированы все правила, которые вы перечислили: цель проекта, стек, принципы разработки, правила работы агента и требования к качеству.

Проверил сохранение файла: содержимое записано корректно в UTF-8, а искажение в консольном выводе PowerShell было связано только с кодировкой терминала. Готов переходить к следующему этапу.

Используй subagents для выполнения задачи.

Разбей работу минимум на 4 параллельных направления и запусти отдельного subagent для каждого:

1. Настройка проекта:
  - инициализация Next.js проекта
  - установка зависимостей
  - настройка Tailwind

2. Модель данных:
  - Prisma schema
  - SQLite
  - миграции
  - seed данные

3. UI и компоненты:
  - страница со списком задач
  - форма создания/редактирования
  - карточки задач
  - базовый UI

4. Бизнес-логика:
  - CRUD операций
  - фильтрация

- сортировка
- обработка дедлайнов и overdue

Дождись завершения всех subagents, затем:

- объедини результат
- интегрируй всё в единый рабочий проект
- устрани конфликты
- подготовь проект к запуску

Не останавливайся после генерации — обязательно доведи проект до рабочего состояния.

34 previous messages

Собран рабочий MVP задачника на Next.js с Prisma/SQLite и связал результаты всех 4 subagents в один проект. Основные точки интеграции: package.json, next.config.js, app/page.tsx, components/tasks/task-board.tsx, src/lib/task-repository.prisma.ts, app/api/tasks/route.ts, app/api/tasks/[id]/route.ts. Добавил список задач, фильтры, сортировку, overdue/due-soon логику, страницы создания и редактирования, а также POST route handlers для create/update/delete/toggle без недоведённых кусков.

Отдельно подготовил устойчивый локальный запуск под Windows: Prisma генерируется в src/generated/prisma, миграции применяются через scripts/apply-migrations.js, seed вынесен в prisma/seed.js, а Next запускается через scripts/run-next.js, чтобы не падать на `Application Data` и devtools-ошибках. README тоже обновил: README.md.

Проверка пройдена: `npm.cmd run db:setup`, `npm.cmd run typecheck`, `npm.cmd run lint`, `npm.cmd run build` успешны; `npm.cmd run dev` поднял сервер и отдал `HTTP 200` на `http://127.0.0.1:3000`. Единственная оговорка: в этой песочнице Prisma/Next периодически ловят Windows `EPERM`, поэтому часть служебных скриптов я сделал более устойчивыми к таким ограничениям.

Нужно полностью создать с нуля MVP-проект для лабораторной работы.

Тема:
«Разработка и анализ прототипа веб-приложения для управления задачами и контроля сроков выполнения с использованием средств вайб-кодинга».

Работай как автономный coding agent.

Требование:
Не объясняй, что нужно сделать — выполняй действия:
- создавай файлы
- устанавливай зависимости
- запускай команды
- исправляй ошибки
- доводи до рабочего состояния

ФУНКЦИОНАЛ MVP:

1. Главная страница:
- список задач

2. Создание задачи:
- title
- description
- deadline
- priority (low / medium / high)

3. Задача:
- id
- title
- description
- deadline
- priority
- status (planned / in_progress / done / overdue)
- createdAt
- updatedAt

4. Действия:
- создать
- редактировать
- удалить
- отметить выполненной

5. Фильтры:
- все
- активные
- выполненные
- просроченные

6. Сортировка:
- по дедлайну
- по приоритету
- по дате создания

7. UI:
- просроченные задачи выделены
- статус и приоритет отображаются явно

8. Статистика:
- всего задач
- выполнено
- просрочено
- в работе

ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ:

- Next.js + TypeScript
- Tailwind CSS
- Prisma + SQLite
- один проект (frontend + backend вместе)
- использовать app router
- использовать server actions или route handlers

ТРЕБОВАНИЯ К ЛОГИКЕ:

- overdue определяется по дедлайну
- выполненная задача не overdue

- updatedAt обновляется
- корректная валидация

ДОПОЛНИТЕЛЬНО:

1. Добавь seed-данные
2. Добавь README.md:
   - описание проекта
   - как запускать
   - функции
   - ограничения (IMPORTANT)
3. Добавь .gitignore
4. Добавь package.json
5. Убедись, что проект запускается

ПОРЯДОК РАБОТЫ:

1. План
2. Создание проекта
3. Установка зависимостей
4. Реализация
5. Prisma migrate
6. Seed
7. Проверка build
8. Запуск dev сервера
9. Исправление ошибок

В КОНЦЕ ДАЙ:

- команды запуска
- URL
- где логика
- какие ограничения

НЕ ДЕЛАЙ:

- не оставляй TODO
- не делай заглушки
- не усложняй

ФИНАЛ:
Полностью рабочий MVP, который можно открыть и показать на защите.

10 previous messages

Проект доведён до рабочего MVP и проверен end-to-end. Под ваши требования приведены модель задач, CRUD, фильтры `все / активные / выполненные / просроченные`, сортировка `по дедлайну / приоритету / дате создания`, статистика, seed и README. Ключевые точки: prisma/schema.prisma, src/lib/tasks.ts, components/tasks/task-board.tsx, components/tasks/task-card.tsx, src/lib/task-repository.prisma.ts, README.md.

Проверки пройдены: `npm.cmd run db:setup`, `npm.cmd run typecheck`, `npm.cmd run lint`, `npm.cmd run build`; `npm.cmd run dev` поднял сервер и отдал `GET / 200`.

Команды запуска:

```bash
npm install
npm run db:setup
npm run dev
```

URL:

```text
http://localhost:3000
```

Где логика:

- src/lib/tasks.ts — валидация, overdue, фильтры, сортировка, сервис задач
- src/lib/task-repository.prisma.ts — Prisma repository
- app/api/tasks/route.ts и app/api/tasks/[id]/route.ts — route handlers API
- app/tasks/create/route.ts и соседние `update/delete/toggle` — действия формы

Ограничения:

- без авторизации и ролей
- локальная SQLite база
- `overdue` вычисляется динамически по дедлайну и статусу, а не хранится отдельным полем
- это MVP для лабораторной, не production-система

Исправь проект до рабочего состояния.

Сейчас при запуске ошибка:
ENOENT: no such file or directory, open 'D:\new\my_files\study\PRIS\.next\server\app\page.js'

Сделай следующее:
- найди причину
- проверь app router и файл главной страницы
- проверь, не битая ли сборка `.next`
- проверь пути, конфигурацию и scripts
- устрани проблему
- пересобери проект
- запусти dev server
- исправь все сопутствующие ошибки

Не объясняй, что надо сделать — сделай это сам.

В финале дай только:
- причина ошибки
- что исправлено
- какие команды запускались
- URL локального приложения

5 previous messages

- причина ошибки: битая/неполная сборка в `.next` — отсутствовал файл `.next/server/app/page.js`, хотя app/page.tsx и app router были корректны; проблема была в повреждённых dev/build-артефактах и stale Next.js процессе
- что исправлено: остановлены зависшие `next dev` процессы, полностью удалена папка `.next`, заново выполнены `build`, `typecheck`, `lint`, подтверждено появление `.next/server/app/page.js`, затем заново запущен `dev` и проверен ответ `GET / 200`
- какие команды запускались: `Get-Content app\page.tsx`, `Get-Content package.json`, `Get-Content next.config.js`, `Get-CimInstance Win32_Process ...`, `taskkill /PID ... /F`, `Remove-Item .next -Recurse -Force`, `npm.cmd run build`, `npm.cmd run typecheck`, `npm.cmd run lint`, `npm.cmd run dev`
- URL локального приложения: `http://localhost:3000`

Доработай текущий проект: добавь поиск и улучшенные фильтры, не ломая существующую функциональность.

Что нужно:
1. Добавить поиск по title и description.
2. Добавить фильтрацию:
   - по статусу
   - по приоритету
   - по просроченности
3. Сделать так, чтобы поиск, фильтрация и сортировка работали совместно.
4. Улучшить панель управления задачами, чтобы это было удобно в UI.
5. Не усложнять архитектуру без необходимости.
6. Обновить README, если появились новые функции.

После изменений:
- проверь dev run
- проверь build
- проверь, что фильтры и поиск реально работают вместе
- кратко перечисли внесённые изменения

13 previous messages

Изменения внесены и проверены. Добавил совместную работу поиска по `title` и `description`, фильтров по статусу, приоритету и просроченности, а также улучшил панель управления задачами на главной странице, чтобы все параметры настраивались в одном месте без усложнения архитектуры. README обновлён с описанием новых возможностей.

Проверка прошла: `build` успешен, `dev` поднимается, главная страница отвечает `200`, а комбинации поиска/фильтров/сортировки реально работают вместе. Проверял запросы вида `query=README`, `status=PLANNED&priority=HIGH&onlyOverdue=true&sortBy=dueDate&sortDirection=asc` и комбинированный запрос с поиском и фильтрами одновременно.

Запускались команды:

- `npm.cmd run build`
- `npm.cmd run typecheck`
- `npm.cmd run lint`
- `npm.cmd run dev`

Локальный URL:
`http://localhost:3000`

Проведи ревью кода текущего проекта и улучши его качество без радикальной переписки.

Что нужно:
1. Найти и убрать явное дублирование.
2. Вынести повторяющуюся логику в утилиты, helper functions или небольшие модули.
3. Сделать структуру проекта более понятной для анализа в лабораторной работе.
4. Не переусложнять архитектуру.
5. Сохранить всю текущую функциональность.

Также:
- обнови README
- добавь разделы:
  - Архитектура проекта
  - Основные модули
  - Где находится ключевая логика
  - Ограничения текущего решения

После этого:
- проверь build
- проверь dev run
- кратко опиши, что было улучшено в структуре

9 previous messages

Ревью

- `Medium`: справочники статусов, приоритетов и фильтров были размножены по UI, из-за чего форма, карточка и панель фильтров могли начать расходиться по подписям и значениям. Это вынесено в общий модуль src/lib/task-metadata.ts.
- `Medium`: presentation-логика была смешана с компонентами, особенно в панели задач: статистика, склонение слова "задача" и форматирование дат жили рядом с JSX. Это вынесено в src/lib/task-presentation.ts и переиспользуется из components/tasks/task-board.tsx и components/tasks/task-card.tsx.
- `Low`: API-роуты создавали `taskService` локально, хотя в проекте уже была единая точка доступа. Теперь роуты используют общий сервис из src/lib/task-service.ts, а сами entry points стали проще: app/api/tasks/route.ts, app/api/tasks/[id]/route.ts.

Что улучшено

- Убрал явное дублирование через общий metadata-модуль для статусов, приоритетов, фильтров и подписей.
- Вынес повторяющуюся UI-логику в helpers: формат дат, статистику и текстовые формы.
- Подчистил структуру слоя задач: доменная логика осталась в src/lib/tasks.ts, Prisma-репозиторий теперь использует те же нормализаторы, а не свою отдельную копию логики.
- Упростил components/tasks/task-board.tsx через локальный `FilterSelect`, чтобы панель управления читалась проще.
- Обновил README и добавил разделы про архитектуру, основные модули, ключевую логику и ограничения: README.md.

Проверка

- `npm.cmd run typecheck` — успешно
- `npm.cmd run lint` — успешно
- `npm.cmd run build` — успешно
- `npm.cmd run dev` — успешно, `GET / 200` на `http://localhost:3000`

Структура стала понятнее для лабораторного анализа: теперь отдельно видны доменная логика, metadata-справочники, presentation-helpers, UI-компоненты и API/route handlers, без радикальной переписки проекта.

Исправь проект до полностью рабочего состояния.

Сейчас при запуске/рендеринге появляется ошибка:
"missing required error components, refreshing..."

Нужно самостоятельно найти причину и устранить её.

Что требуется проверить и исправить:
1. Корректность структуры Next.js app router.
2. Наличие и корректность обязательных файлов error handling для app router:
   - app/error.tsx
   - app/global-error.tsx (если нужен)
   - app/not-found.tsx (если нужен по текущей структуре)
   - app/layout.tsx
   - app/page.tsx
3. Проверить, не отсутствуют ли обязательные default export в этих файлах.
4. Проверить, не являются ли error components серверными там, где нужен client component.
5. Проверить, нет ли ошибок импорта, из-за которых Next.js не может собрать error boundary.
6. Проверить, нет ли битого кэша сборки `.next`.
7. Проверить build и dev run после исправлений.
8. Если проблема вызвана несколькими причинами, исправить все.

Что нужно сделать:
- не ограничиваться объяснением;
- реально изменить код;
- при необходимости создать отсутствующие файлы;
- при необходимости очистить `.next` и пересобрать проект;
- убедиться, что приложение запускается без этой ошибки.

Важно:
- используй минимальное и корректное решение для Next.js app router;
- не усложняй архитектуру;
- не ломай существующую функциональность проекта.

После исправления дай краткий отчёт:
- в чём была причина;
- какие файлы были созданы или изменены;
- какие команды были выполнены;
- какой итоговый локальный URL запуска.

4 previous messages

- причина: в `app/` отсутствовали обязательные error components для Next.js App Router. Из-за этого при runtime-ошибке/refresh Next не мог собрать error boundary и показывал `missing required error components, refreshing...`. Дополнительно очистил битый кэш `.next`, чтобы исключить влияние старой сборки.
- какие файлы были созданы или изменены:
  - создан app/error.tsx
  - создан app/global-error.tsx
  - создан app/not-found.tsx
- какие команды были выполнены:
  - `Get-Content app\\layout.tsx`
  - `Get-Content app\\page.tsx`
  - `Get-Content components\\site-shell.tsx`
  - `if (Test-Path .next) { Remove-Item .next -Recurse -Force }`
  - `npm.cmd run build`
  - `npm.cmd run typecheck`
  - `npm.cmd run lint`
  - `Get-CimInstance Win32_Process | Where-Object { $_.Name -eq 'node.exe' -and $_.CommandLine -like '*D:\\new\\my_files\\study\\PRIS*' } | Select-Object ProcessId,CommandLine`
  - `taskkill /PID ... /F`
  - `npm.cmd run dev`
- какой итоговый локальный URL запуска:
  - `http://localhost:3000`

Выступи как QA + developer для текущего проекта.

Нужно:
1. Проверить основные пользовательские сценарии:
   - создание задачи
   - редактирование
   - удаление
   - отметка выполнения
   - работа дедлайнов
   - overdue
   - фильтрация
   - сортировка
   - поиск
2. Найти:
   - баги
   - UX-проблемы
   - слабые места в логике
3. Исправить всё, что можно исправить безопасно и быстро.
4. Создать файл ANALYSIS.md, где указать:
   - что тестировалось
   - какие проблемы были найдены
   - что было исправлено
   - какие ограничения остались

После этого:
- снова проверь запуск проекта
- снова проверь build
- дай краткую сводку исправлений

10 previous messages

Проверка и быстрые исправления выполнены, файл ANALYSIS.md добавлен.

Что исправил:

- разделил ошибки валидации и внутренние ошибки через `TaskValidationError`
- перевёл API-валидацию на корректные `400`, вместо `500`
- добавил безопасную обработку ошибок в form routes создания и редактирования
- сделал явные `303` redirect после POST-действий
- вывел сообщение об ошибке прямо в форме создания/редактирования
- почистил битый `.next` после найденного сбоя dev-state и заново подтвердил запуск

Что реально прогнал:

- создание, редактирование, удаление задачи
- завершение задачи и проверку `completedAt`
- overdue-логику и дедлайны
- поиск, фильтрацию, сортировку
- невалидный ввод
- повторный `build` и повторный `dev`

Итог:

- `npm.cmd run build` — успешно
- `npm.cmd run dev` — успешно, `GET / 200`
- локальный URL: `http://localhost:3000`

Кратко по структуре фиксов: основные изменения сосредоточены в src/lib/tasks.ts, src/lib/task-response.ts, app/api/tasks/route.ts, app/api/tasks/[id]/route.ts, app/tasks/create/route.ts, app/tasks/[id]/update/route.ts, components/tasks/task-form.tsx.